

DL-06-T9

Chatbot with Database Search

By [Sunain Mushtaq](#)

Task Summary

DL-06-T9 focused on implementing the first technical MCP-style tool layer for the DiscountMate chatbot. The work gives the future LLM agent a structured way to search products, retrieve product details, fetch current prices, and complete a price comparison using real DiscountMate data instead of model-generated facts.

Accomplishments

- Created a dedicated chatbot workspace inside Backend/ml-service for future agent, tool, schema, service, and evaluation code.
- Designed shared JSON response envelopes and product/price schemas so tool outputs are predictable for LangGraph and frontend integration.
- Implemented MCP-style tools for `search_products`, `get_product_details`, `get_current_prices`, and `compare_prices`.
- Built a MongoDB-backed repository that reads from the `current_products`, `product_pricings`, and `categories` collections.
- Added ambiguity handling for broad searches, returning candidate products and a clarification state when a direct comparison is not reliable.
- Added local fake-data and real MongoDB smoke tests for repeatable terminal validation.

Implementation Details

Tool Layer

- `search_products`: finds product candidates using product name, brand, pack size, category, and limit inputs.
- `get_product_details`: returns display-ready product metadata including name, brand, category, pack size, GTIN, image, and current prices.
- `get_current_prices`: retrieves valid current retailer prices for a resolved product ID.
- `compare_prices`: combines product resolution and price retrieval, sorts prices cheapest first, and returns the cheapest retailer when available.

Backend Access

- Flask endpoints were added under `/api/chatbot/tools` for direct ML service access.
- Express proxy routes were added under `/api/ml/chatbot/tools` so backend/frontend callers can use the existing API gateway path.
- The implementation keeps business data retrieval inside tools and services, while leaving the future LLM agent responsible only for intent, entity extraction, tool selection, and response wording.

Testing Evidence

- Python syntax checks passed for the chatbot package and Flask app file.
- Node syntax checks passed for the Express controller and router updates.
- Fake-data test passed for all four tools and confirmed the price comparison flow.
- Real MongoDB smoke test connected successfully, searched products, fetched prices, and returned a successful comparison result for a resolved milk product.

Outcome

DL-06-T9 is now ready as an initial technical contribution. The chatbot has a working database-backed tool layer that can be called directly, tested in the terminal, and integrated into the later DL-06-T8/DL-06-T11 LLM agent workflow.